Module IV

DIVIDE AND CONQUER.

Divide and conquer is a top-down technique for designing algorithms that consists of dividing the problem into smaller subproblems hoping that the solution of the subproblems are easier to find and then composing the partial solutions into the solution of the original problem.

Divide and conquer algorithms have 3 parts:

- Divide the problem into a number of subproblems that are smaller instances of the same problem.
- Conquer the subproblems by solving them recursively. If they are small enough, solve the subproblems as base case.
- Combine the solutions to the subproblem into the solution for the original problem.

→ The Control Abstraction

```
Algorithm DAC (P)
{
    if P is small, then return S(P)
    else
    {
        Divide P into smaller sub instances
        P1, P2, ... Pk   k>1
        Apply DAC on each Pi
        return Combine (DAC(P1), DAC(P2) ... DAC(Pk))
    }
}
```

- Control Abstraction defines the flow of control of any algorithm
- S(P) is some function that is applied on the smallest instance of P to compute the solution.
- $P_1, P_2 \ldots P_k$ are solved by recursive application of divide and conquer. Combine is a function that determines the solution to P using the solution to k subproblems.

The solution to above problem is described by the recurrence, assuming size of P denoted by n

2

$$T(n) = \begin{cases} g(n) & n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & n \text{ is large} \end{cases}$$

2-way Merge Sort

Divide : Partition the array into two sublists S_1 and S_2 with $n/2$ elements each.

Conquer : sort sublist S_1 and S_2.

Combine : Merge S_1 and S_2 into a unique sorted group.

```
Algorithm MergeSort (low, high)
{
    if (low < high)
    {
        mid = [(low+high)/2]   // dividing position
        MergeSort (low , mid)
        MergeSort (mid+1 , high)
        Merge (low, mid, high) ;  // combining solution
    }
}
```

→ a[low: high] is a global array to be sorted

3

```
Algorithm Merge (low, mid, high)
{
    h = low ; i = low ; j = mid + 1 ;
    while ((h ≤ mid) and (j ≤ high)) do
    {
        if (a[h] ≤ a[j]) then
        {
            b[i] = a[h] ;
            h = h+1 ;
        }
        else
        {
            b[i] = a[j]
            j = j+1
        }
        i = i+1 ;
    }
    if (h > mid) then
        for k = j to high do
        {
            b[i] = a[k] ;   i = i+1 ;
        }
    else
        for k = h to mid do
        {
            b[i] = a[k] ;  i = i+1 ;
        }
    for k = low to high do a[k] = b[k] ;
}
```


4

Complexity Analysis

$$T(n) \begin{cases} c & n \text{ is small} \\ 2T(n/2) + n & n \text{ is large} \end{cases}$$

$$T(n) = 2T(n/2) + 2.$$

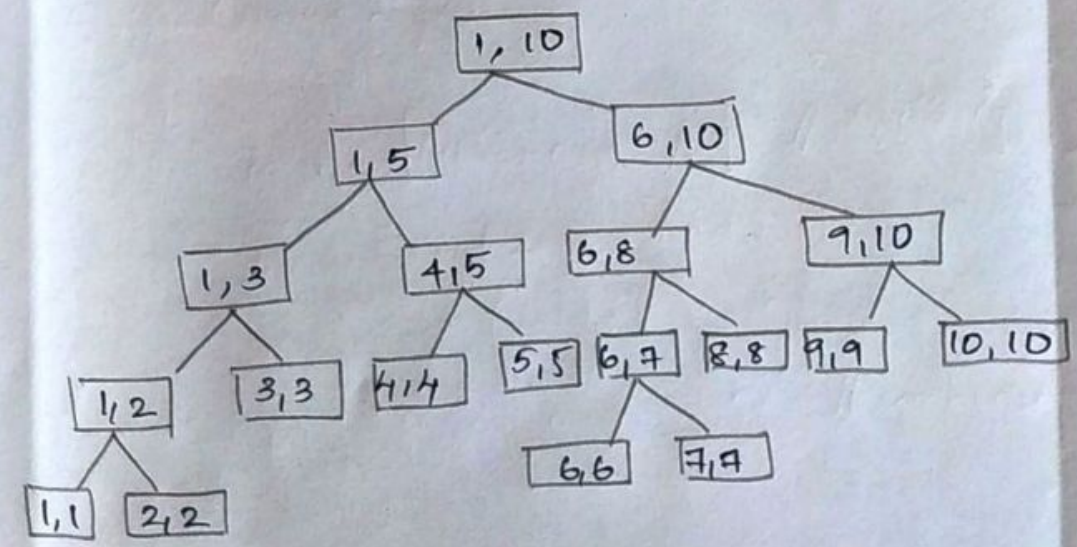
According to master's theorem,

$a = 2$
$b = 2$

$$n^{\log_b^a} = n^1 = f(n)$$

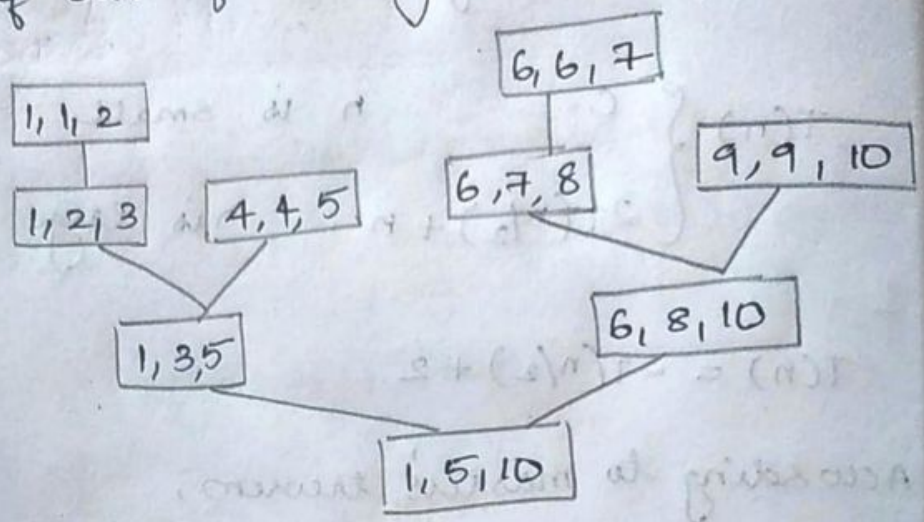
$$\therefore \boxed{T(n) = \Theta(n \log n)}$$

⇒ Tree of calls for merge sort



5

⇒ Tree of calls for merge



Q. Solve using 2-way merge sort.

10	2	3	7	6	4	8
0	1	2	3	4	5	6.

$mid = \frac{0+6}{2}$
$= 3$

10	2	3	7	6	4	8
0			3	4		6

$mid = \frac{0+3}{2}, \frac{4+6}{2}$
$1, 5$

10	2	3	7	6	4	8
0	1	2	3	4	5	6

$mid = \frac{0+1}{2}, \frac{2+3}{2}, \frac{4+5}{2}$
$= 0, 2, 4$

10	2	3	7	6	4
0	1	2	3	4	5

merge(0,0,1), merge(2,2,3)
merge(4,4,5)

2	10	3	7	4	6
0	1	2	3	4	5

merge(0,1,3) merge(4,5,6)

2	3	7	10	4	6	8
0	1	2	3	4	5	6

6

merge (0,3,6)

2	3	4	6	7	8	10
0	1	2	3	4	5	6

$0+8 = 8/4$

Q. 8, 16, 23, 42, 36, 7, 2, 1, 2

mid $= \frac{0+8}{2} = 4$

8	16	23	42	36
0	1	2	3	4

7	2	1	2
5	6	7	8

mid $= \frac{0+4}{2}, \frac{5+8}{2}$

2, 6

8	16	23
0	1	2

42	36
3	4

7	2
5	6

1	2
7	8

$\frac{0+2}{2}, \frac{3+4}{2}, \frac{5+6}{2}, \frac{7+8}{2}$

1, 3, 5, 7

8	16	23	42	36	7	2	1	2
0	1	2	3	4	5	6	7	8

mid $= \frac{0+1}{2} = 0$

merge (0,1,1)

8	16

merge (0,1,2)

8	16	23
0	1	2

36	42
3	4

2	7
5	6

1	2
7	8

merge (3,4,4)

merge (5,6,6)

merge (7,8,8)

merge(0,3,4) merge(5,6,8)

8	16	23	36	42
0	1	2	3	4

1	2	2	7
5	6	7	8

merge (0,4,8)

1	2	2	7	8	16	23	36	42
0	1	2	3	4	5	6	7	8

7